XDP-L2-Guard: Technical Reference Manual

High-Performance Layer-2 & Layer-3 Traffic Orchestration with eBPF/XDP

Krzysztof Taraszkiewicz
Józef Sztabiński

June 16, 2026

Abstract

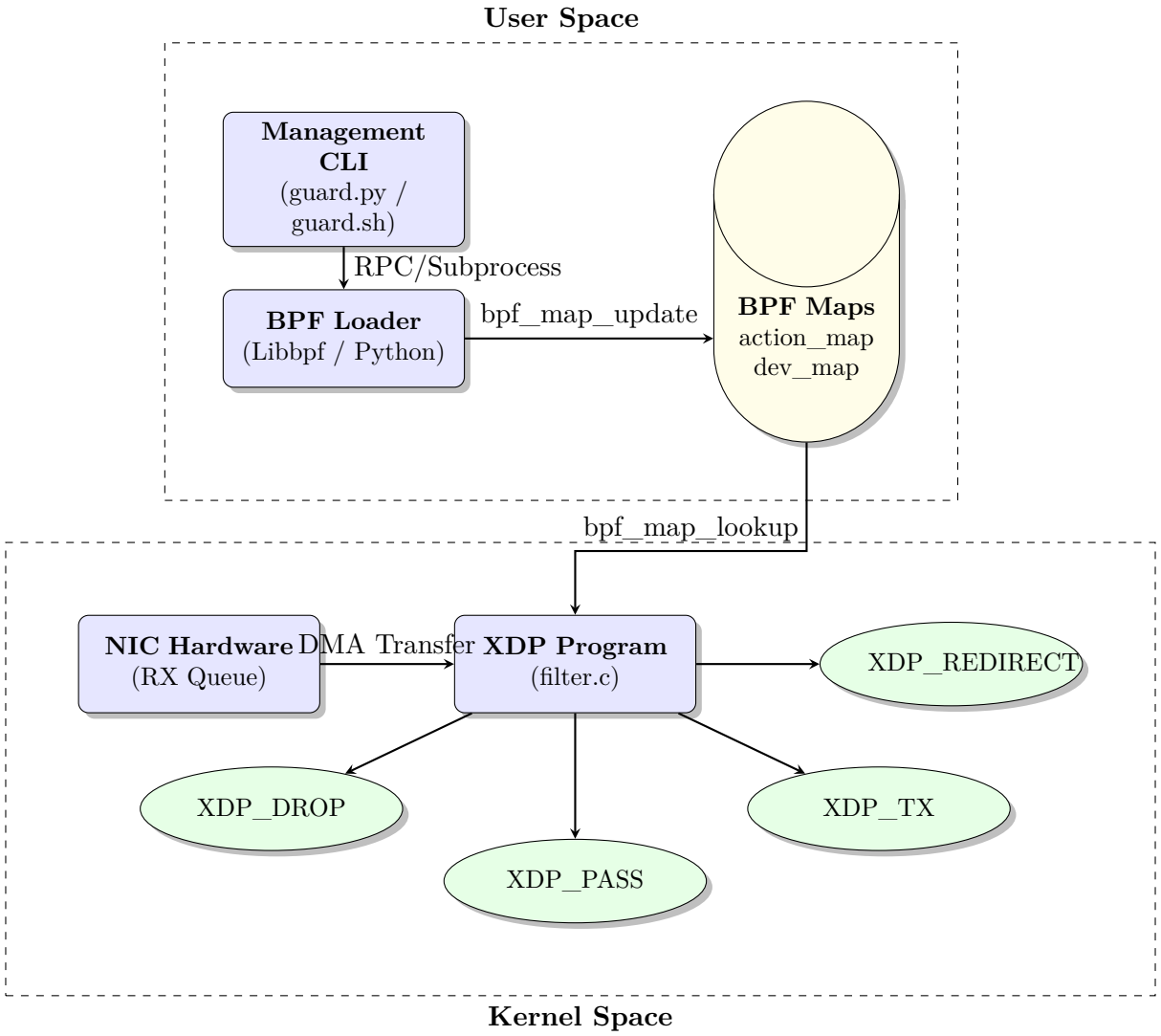
XDP-L2-Guard is a high-performance, programmable network orchestration filter leveraging the Linux eXpress Data Path (XDP). Operating directly inside the Network Interface Card (NIC) driver hook, it provides sub-microsecond packet processing capabilities for Layer-2 and Layer-3 traffic. This enables wire-speed dropping, packet redirection, hairpinning, and state-aware Network Address Translation (NAT) before the Linux kernel allocates socket buffers (`sk_buff`). This manual serves as an exhaustive, low-level technical reference covering C-level memory structures, Python CO-RE management, checksum algorithms, and deployment strategies.

Contents

1 System Architecture and Paradigm

The system is strictly architected around the paradigm of control/data plane separation. The **Data Plane** is written in restricted C, compiled via Clang/LLVM into eBPF bytecode, and executes securely within the kernel context. The **Control Plane** utilizes Python 3 to orchestrate the lifecycle of the eBPF program, utilizing `iproute2` and `bpftool` to manage asynchronous state updates.

1.1 Architectural Component Diagram



2 Data Plane Implementation (eBPF/XDP)

The data plane code (`src/data_plane/filter.c`) hooks into the NIC's NAPI poll loop via the `SEC("xdp")` macro. The function `xdp_drop_logic` acts as the primary ingress orchestrator.

2.1 Memory Layout and Pointer Validation

eBPF programs operate in a verified environment. Before the verifier allows access to packet memory, the code must prove that the memory bounds are safe. The program receives a `struct xdp_md` context, providing DMA mapped pointers:

```
void *data = (void *)(long)ctx->data;
void *data_end = (void *)(long)ctx->data_end;
```

We utilize a custom `BOUNDS_CHECK` macro defined in `headers.h`:

```
#define BOUNDS_CHECK(pointer, type, data_end) \
    do { \
        if ((void *)(pointer) + sizeof(type) > (void *)(data_end)) \
            return XDP_PASS; \
    } while(0)
```

The parsing pipeline proceeds strictly layer by layer:

1. **Layer 2 (Ethernet)**: Cast `data` to `struct ethhdr`. Verify bounds. Check if `eth->h_proto == bpf_htons(ETH_P_IP)`. If not, bypass to kernel via `XDP_PASS`.
2. **Layer 3 (IPv4)**: Cast `data + sizeof(struct ethhdr)` to `struct iphdr`. Verify bounds. Extract the Destination IP (`ip->daddr`).

2.2 Action Map Schema

The extracted IP is used as a lookup key in the primary BPF hash map, `action_map`. The map expects an IPv4 address in network byte order (`__u32`) and returns a `struct action_cfg`:

```
struct action_cfg {
    __u32 target;           // 0: PASS, 1: DROP, 2: TX, 3: REDIRECT, 4:
    NAT
    __u32 new_ip;           // Destination IP for NAT scenarios
    __u32 ifindex;          // OS Interface index for REDIRECT
    __u64 dropped_packets;  // Atomic performance counter
};
```

2.3 Low-Level Action Semantics

Depending on the retrieved `target`, the kernel executes one of the following routines:

2.3.1 XDP_DROP

The simplest and fastest action. The NIC driver drops the frame and immediately recycles the DMA buffer. The code atomically increments the global array map `total_dropped` and the rule-specific `dropped_packets`.

2.3.2 XDP_TX

`XDP_TX` bounces the packet back out of the interface it arrived on. In our implementation, this is used for advanced ICMP manipulation. If an `ICMP_ECHO` (Ping) is detected:

1. The type is rewritten to `ICMP_ECHOREPLY` (0).
2. **Checksum Math**: Because Type 8 changed to Type 0, the 16-bit 1's complement checksum must be incrementally updated by adding `0x0800`.
3. MAC and IP addresses are swapped utilizing `__builtin_memcpy` for optimal inline compilation.

2.3.3 XDP_REDIRECT & ACTION_NAT (DNAT)

`ACTION_NAT` performs Destination Network Address Translation directly in hardware.

```
__u32 old_daddr = ip->daddr;
__u32 new_daddr = cfg->new_ip;
__u32 csum = ~bpf_ntohs(ip->check) & 0xFFFF;
csum += (~(old_daddr & 0xFFFF) & 0xFFFF) + (new_daddr & 0xFFFF);
csum += (~(old_daddr >> 16) & 0xFFFF) + (new_daddr >> 16);
while (csum >> 16) csum = (csum & 0xFFFF) + (csum >> 16);
ip->check = bpf_htons(~csum & 0xFFFF);
```

This bitwise logic incrementally updates the IP header checksum without requiring a full packet recalculation, ensuring sub-microsecond latency. `XDP_REDIRECT` follows the exact same logic but concludes by forwarding the frame to an interface indexed in `dev_map`.

3 Control Plane: Orchestration and Polling

The Control Plane comprises two primary Python scripts: `loader.py` for lifecycle management and `guard.py` for CLI rule definition.

3.1 The Loader Engine (`loader.py`)

The loader handles Ahead-Of-Time (AOT) compilation and kernel attachment via the `iproute2` toolchain.

- **Compilation**: Triggers `make` to compile `filter.c` into the ELF object `filter.o`.
- **Mode Selection**: Detects whether to mount in high-performance native mode (`xdpdrv`) or generic software mode (`xdpgeneric`).
- **Attachment**: Uses `ip -force link set dev <interface> xdpdrv obj filter.o sec xdp`. The `-force` flag guarantees atomic overwrites of zombie programs.
- **Asynchronous Polling**: Operates an infinite loop (`time.sleep(1)`) that invokes `bpftool -j map dump name action_map`. It decodes the JSON payload, unpacks the byte streams, and translates the raw metrics into human-readable logs.

3.2 Management CLI (`guard.py` & `guard.sh`)

The project provides two identical `iptables`-like interfaces to manipulate the active BPF maps via `bpftool`:

- **`src/control_plane/guard.py`**: The primary Python implementation using `argparse`.
- **`scripts/guard.sh`**: A pure Bash alternative utilizing `awk` and `sed` for environments where Python is unavailable or undesired.

Both scripts share the exact same syntax and arguments.
CLI Syntax Examples:

```
# Append a standard DROP rule for a malicious actor (Python version)
sudo python3 guard.py --append --destination 192.168.1.50 --jump DROP

# Establish a hardware-level NAT translation (Bash version)
sudo ./guard.sh -A -d 10.0.0.100 -j NAT --to-destination 10.0.0.250
```

```
# Redirect traffic arriving for 10.0.0.5 to another Virtual Ethernet
sudo ./guard.sh -A -d 10.0.0.5 -j REDIRECT --oif veth1

# List all active rules dynamically pulled from kernel memory
sudo python3 guard.py --list
```

Internally, both scripts resolve system interfaces (e.g., `veth1`) to integer `ifindex` values via `/sys/class/net/<iface>/ifindex` and format the C-struct byte arrays (either via Python's `struct.pack` or Bash's `printf`) to be injected into the kernel via `bpftool map update`.

4 Action Showcase and Testing Examples

To demonstrate and validate the capabilities of XDP-L2-Guard, the `scripts/example/` directory provides an interactive playground utilizing Linux Network Namespaces (netns).

4.1 Network Topology Setup

The `setup.sh` script initializes a simulated 3-node network environment:

- **`ns1` (Filter/Router)**: Assigned IP `10.0.0.1`. The eBPF object is attached to virtual interfaces `v1-2` and `v1-3`. IPv4 forwarding is enabled.
- **`ns2` (Sender)**: Assigned IP `10.0.0.2`. Acts as the source of traffic.
- **`ns3` (Target)**: Assigned IP `10.0.0.3` in promiscuous mode to observe redirected packets.

4.2 Applying Actions

The `set_action.sh` script uses `guard.sh` to dynamically apply rules for traffic originating from `ns2`:

```
# Terminal 1: Run the setup and keep the XDP filter active
sudo ./setup.sh

# Terminal 2: Test different XDP actions in real-time
./set_action.sh drop      # Traffic from ns2 is silently dropped
./set_action.sh redirect  # Traffic from ns2 is bounced to ns3
./set_action.sh tx        # Hairpin ICMP echo (Ping responds instantly)
./set_action.sh nat       # Destination IP is rewritten to 10.0.0.3
```

This setup proves that complex Layer-2 (REDIRECT) and Layer-3 (NAT) operations occur entirely within the eBPF layer, completely bypassing the conventional routing logic in `ns1`.

5 Operational DevOps & Diagnostics

5.1 Performance Tuning & Real-World Limits

To achieve theoretical maximum throughput (10-40 Gbps), several hardware-level optimizations are mandatory.

- **Native Mode (`xdpdrv`) vs. Generic Mode (`xdpgeneric`)**: Native mode runs the eBPF program directly inside the NIC driver, before the kernel allocates any memory. Generic mode runs later in the stack. Native mode is essential for flood protection.
- **Disable Hardware Offloads**: Offloads like Generic Receive Offload (GRO) aggregate packets before XDP sees them, bypassing the filter. Disable via: `sudo ethtool -K eth0 gro off gso off rxvlan off`.

- **CPU Pinning**: Ensure XDP IRQs are pinned to dedicated physical cores using `smp_affinity`.

5.1.1 Case Study: Intel NIC Native Mode Testing

During real-world load testing utilizing multiple independent attacking machines flooding an Intel network interface, our goal was to leverage Native XDP mode (`xdpdrv`) for maximum hardware acceleration. Ultimately, we encountered low-level hardware configuration challenges rather than hitting the computational limits of the host CPU.

While we were unable to perfectly align the NIC driver configuration to achieve wire-speed processing in this specific test, we recognize this as a critical area for future development. We are fully aware of the massive performance potential that native hardware execution offers. We were exceptionally close to a working native implementation, and resolving these hardware-specific configuration nuances remains a high-priority milestone for the next evolution of XDP-L2-Guard.

5.2 Diagnostic Matrix

Symptom	**Root Cause Analysis**	**Remediation**
`libbpf: load fail`	eBPF Verifier rejected code bounds.	Inspect `dmesg`. Ensure `BOUNDS_CHECK` covers all accessed memory.
Rules applied, no drop	Attached to wrong interface or mode.	Check with `ip link show`. Ensure XDP is attached to the physical interface, not a bridge.
`EBUSY` during attach	Another XDP program is already loaded.	Use the `-force` flag in `ip link set dev <iface> xdp obj ...`
High SoftIRQ CPU	Lock contention on `total_dropped`.	Refactor map to `BPF_MAP_TYPE_PERCPU_ARRAY`.
Packets bypass filter	Hardware offloading is active.	Disable GRO/LRO using `ethtool -K <iface> gro off`.
BPF map creation fails	`RLIMIT_MEMLOCK` is too low.	Increase memory limits: `ulimit -l unlimited`.
`XDP_TX` fails silently	Checksum recalculation is incorrect.	Verify 1's complement math. Ensure MAC swapping logic is correct.